Brushed DC Motor Speed Estimation Using a State Space and Neural Network Hybrid Approach

Team members: Mohammad Elmougey, Heinz Veintimilla

1. Problem Statement

Mechanical encoders are widely used for providing a system with the speed a motor is spinning at as feedback for a control system. Motor encoders are generally expensive and require selecting a specific model that can properly couple with the motor of choice. Integrating encoders can be challenging due to space constraints as well as harsh environmental conditions. Vibration and mechanical shock, extreme temperatures, contamination and moisture are all environmental factors that can impact the accuracy of encoder readings. For these reasons it is desirable to have a way of measuring motor speed for control applications without the use of a mechanical encoder.

2. Motivation

For our Capstone project we are designing an electric wheelchair controlled with subtle head tilts for users paralyzed from the neck down. In order to make driving the wheelchair a pleasant experience for the user, a control system will be used to gradually accelerate the wheelchair to a desired speed. Full state feedback is a common control algorithm which requires knowing all state variables in a system. For controlling a motor the state variables are the motors rotational speed and current supplied to the motor. Our project will include motor current sensing but due to the limitations of mechanical encoders mentioned in the problem statement, a sensorless motor speed estimation approach will be taken. To estimate rotational speed the state-space of a brushed DC motor will be used alongside a Neural Network trained on simulation data. The state-space captures the electro-mechanical dynamics of the motor (torque, resistance, inductance etc..) while the neural network compensates for the non-linear disturbances (friction torque, motor ripple etc..) that would otherwise be difficult to model with a state-space.

3. Proposed Dataset

Data will be collected in matlab using simulink. The brushed DC motor system will be modelled in simulink and include mechanical and electrical dynamics. Simulated dynamics include:

Mechanical Dynamics

- Rotor inertia
- Coulomb friction torque
- Viscous friction
- Rotor damping
- Rolling resistance of attached 24-inch wheel

Electrical Dynamics

- Armature Inductance
- Armature Resistance

Sensor readings (current and speed) will mix with a white noise generator to simulate distortion in a practical scenario. The simulation runs for 40 seconds and samples at 1 millisecond meaning that 40,000 test examples will be generated per run. Supervised learning will be used as each test example will have the inputs to the model as well as the desired output (motor speed obtained using a rotational motion sensor in simulink). Two 40,000 sample datasets will be collected; one for training and the other for testing. The datasets are a series of step functions for the duration of the simulation runtime. Each step is a reference speed the motor controller attempts to bring the true speed of the motor to.

If time permits, we will attempt to collect data of a real life system with variable load, current, and speed sensing to train the model on a practical dataset.

4. Planned Approach

Our approach will be to provide the measured current of the motor (from a current sensor) along with the current and angular velocity predicted by the state space equation of the system to be mapped to a value that more confidently estimates the actual angular velocity of the motor shaft. The state space model’s estimates can be corrected based on the measured current of the motor acting in a similar fashion to the kalman gain in a kalman filter. Due to the nature of reading sensor values for a motor system, the architecture of the model will need to be designed for time series based data. This means the model's learning task will be sequence-to-sequence or in other words both the inputs and outputs will be time series based data. Regression models are

commonly used when continuous (time series) data needs to be output. Two foundational options for regression models include polynomial regressions and neural network regressions. Polynomial regressions are a one shot equation with trainable weights and biases based on the number of features at the input. By increasing the degree of the polynomial more complex patterns can be learned at the risk of overfitting the training data. A lower degree polynomial will learn more general patterns at the cost of potentially underfitting data. Neural network regressions on the other hand, can learn more complex patterns by having several layers of neurons. The neural network being able to pick up subtler non-linear patterns comes at the cost of computational complexity compared to a polynomial regressor. Because the goal of this application would be to eventually run this model on an edge device with constrained processing a polynomial regression model will be used. The degree of the polynomial will be swept from 2nd - 5th order to find the optimal degree for this application.

To train our polynomial regression model we will use and compare two different approaches: 1) an iterative approach and 2) a closed form solution approach. If the training parameters are linear (which is the case for our model) the ordinary least squares (OLS) training method can be used. This is similar to the Moore-Penrose rule in that an exact set of weights can be determined mathematically. For the iterative approach, stochastic gradient descent will be used and compared with the closed form results.

Having 4 different degree polynomial models in our sweep along with 2 types of training means we will have a pool of 8 trials to compare results and select the one that performs the best.

4. Evaluation Metrics

- **Mean Squared Error (MSE):** Penalizes large errors (outliers)
- **Root Mean Squared Error (RMSE):** Also used for detecting outliers as the motor speed reading will be sensitive to them. Having outliers leading to significantly larger errors, means huge swings in the PID controller that can cause the system to operate in an adhoc manner.